SELECT & Filtering

SQL Level 2 - Concepts, Examples and Practice

All examples use MySQL syntax and the Students table below

Topics covered

- SELECT, FROM, AS, DISTINCT
- WHERE and comparison operators (=, <>, >, <, >=, <=)
- AND, OR, NOT, IN, NOT IN, BETWEEN
- LIKE, NOT LIKE and wildcards % and _
- ORDER BY, ASC, DESC, LIMIT, OFFSET
- 15 practice questions with solutions and outputs

Every topic has a definition, syntax, **two examples**, the real output for each example, and an explanation.

Setup: Sample Table

Run this once. All examples and outputs in this book use these 12 rows.

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name       VARCHAR(50),
    age        INT,
    course     VARCHAR(50),
    marks      INT
);

INSERT INTO Students VALUES
(1,  'Aarav',   21, 'Data Science',    82),
(2,  'Priya',   19, 'Python',          74),
(3,  'Ananya',  20, 'Data Science',    91),
(4,  'Kiran',   22, 'Java',            65),
(5,  'Sneha',   18, 'Python',          58),
(6,  'Rohan',   23, 'Web Development', 77),
(7,  'Divya',   24, 'Data Science',    69),
(8,  'Arjun',   25, 'Java',            88),
(9,  'Meena',   19, 'Python',          61),
(10, 'Vikram',  21, 'Web Development', 45),
(11, 'Anjali',  22, 'Data Science',    95),
(12, 'Sanjana', 20, 'Python',          72);
```

Students (sample data for Level 2)

student_id	name	age	course	marks
1	Aarav	21	Data Science	82
2	Priya	19	Python	74
3	Ananya	20	Data Science	91
4	Kiran	22	Java	65
5	Sneha	18	Python	58
6	Rohan	23	Web Development	77
7	Divya	24	Data Science	69
8	Arjun	25	Java	88
9	Meena	19	Python	61
10	Vikram	21	Web Development	45
11	Anjali	22	Data Science	95
12	Sanjana	20	Python	72

Figure 1: The Students table



Figure 2: Parts of a SELECT query, and the order MySQL actually runs them

1. SELECT

What it is: `SELECT` chooses *which columns* to show. It only reads data and never changes the table.

Syntax

```
SELECT column1, column2 FROM table_name;
SELECT * FROM table_name;   -- * = all columns
```

Example 1

```
SELECT * FROM Students;
```

Output (12 rows):

student_id	name	age	course	marks
1	Aarav	21	Data Science	82
2	Priya	19	Python	74
3	Ananya	20	Data Science	91
4	Kiran	22	Java	65
5	Sneha	18	Python	58
6	Rohan	23	Web Development	77
7	Divya	24	Data Science	69
8	Arjun	25	Java	88
9	Meena	19	Python	61
10	Vikram	21	Web Development	45
11	Anjali	22	Data Science	95
12	Sanjana	20	Python	72

Explanation: The star means every column, and with no WHERE every row comes back.

Example 2

```
SELECT name, marks FROM Students;
```

Output (12 rows):

name	marks
Aarav	82
Priya	74
Ananya	91
Kiran	65
Sneha	58
Rohan	77
Divya	69
Arjun	88
Meena	61
Vikram	45
Anjali	95
Sanjana	72

Explanation: Only the two columns you list are returned. Naming columns is faster and clearer than `*`, so prefer it in real projects.

2. FROM

What it is: `FROM` says *which table* the data comes from. MySQL reads this clause first.

Syntax

```
SELECT columns FROM table_name;
SELECT columns FROM database_name.table_name;
```

Example 1

```
SELECT name FROM Students;
```

Explanation: Reads the table *Students* and returns its *name* column (12 names: Aarav, Priya, Ananya, ... Sanjana).

Example 2

```
SELECT name, course FROM CollegeDB.Students;
```

Explanation: Writing `database.table` picks the table from a specific database. This is useful when you have not run `USE CollegeDB;` or when you query two databases at once.

3. AS (Alias)

What it is: AS gives a column or a table a temporary, friendlier name. The real table is not changed.

Syntax

```
SELECT column AS new_name FROM table_name AS t;
```

Example 1

```
SELECT name AS student_name, marks + 5 AS bonus_marks
FROM Students
WHERE student_id <= 3;
```

Output (3 rows):

student_name	bonus_marks
Aarav	87
Priya	79
Ananya	96

Explanation: `name` is shown as *student_name*, and the calculated value `marks + 5` gets the label *bonus_marks*. Without AS, the calculated column would have an ugly title like *marks + 5*.

Example 2

```
SELECT s.name, s.course
FROM Students AS s
WHERE s.age < 19;
```

Output (1 row):

name	course
Sneha	Python

Explanation: `s` is a short nickname for the table, so you can write `s.name`. Table aliases save typing, and they become essential when you join tables later.

4. DISTINCT

What it is: `DISTINCT` removes duplicate rows from the result so each value appears only once.

Syntax

```
SELECT DISTINCT column FROM table_name;
```

Example 1

```
SELECT DISTINCT course FROM Students;
```

Output (4 rows):

course
Data Science
Python
Java
Web Development

Explanation: The table has 12 students but only 4 different courses. DISTINCT shows each course once.

Example 2

```
SELECT DISTINCT course, age
FROM Students
WHERE course = 'Python';
```

Output (3 rows):

course	age
Python	19
Python	18
Python	20

Explanation: With two columns, DISTINCT removes rows where the *whole combination* repeats. Priya and Meena are both (Python, 19), so that pair appears once.

5. WHERE

What it is: `WHERE` filters *rows*. Only rows where the condition is true are returned.

Syntax

```
SELECT columns FROM table_name WHERE condition;
```

Example 1

```
SELECT name, course FROM Students
WHERE course = 'Java';
```

Output (2 rows):

name	course
Kiran	Java
Arjun	Java

Explanation: Only rows whose course is exactly *Java* pass the filter. Text values go in single quotes.

Example 2

```
SELECT name, marks FROM Students
WHERE marks >= 90;
```

Output (2 rows):

name	marks
Ananya	91
Anjali	95

Explanation: Numbers need no quotes. Students with marks 90 or more are returned (Ananya 91, Anjali 95).

6. Comparison Operators

What they are: Symbols used in a WHERE condition to compare a column with a value.

Operator	Meaning	Example
=	Equal to	age = 20
<> or !=	Not equal to	course <> 'Java'
>	Greater than	marks > 70
<	Less than	age < 20
>=	Greater than or equal to	marks >= 90
<=	Less than or equal to	marks <= 58

Example 1

```
SELECT name, marks FROM Students
WHERE marks >= 88;
```

Output (3 rows):

name	marks
Ananya	91
Arjun	88
Anjali	95

Explanation: >= includes 88 itself, so Arjun (88) is in the result as well as Ananya and Anjali.

Example 2

```
SELECT name, marks FROM Students
WHERE marks <= 58;
```

Output (2 rows):

name	marks
Sneha	58
Vikram	45

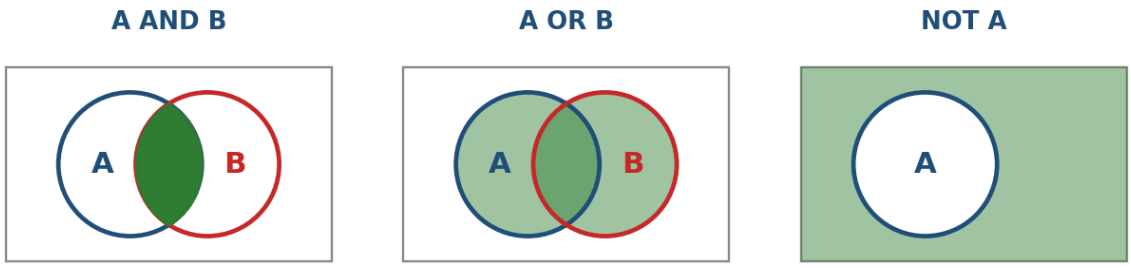
Explanation: <= includes 58, so Sneha (58) is returned along with Vikram (45). Note: in MySQL a single = is used for comparison (not ==).

7. AND

What it is: AND returns a row only if **all** conditions are true.

Syntax

```
WHERE condition1 AND condition2
```



Green = rows that are returned

Figure 3: AND, OR and NOT as diagrams (A and B are two conditions)

Example 1

```
SELECT name, course, marks FROM Students
WHERE course = 'Data Science' AND marks > 90;
```

Output (2 rows):

name	course	marks
Ananya	Data Science	91
Anjali	Data Science	95

Explanation: A student must be in Data Science *and* have marks above 90. Aarav (82) and Divya (69) fail the second test, so they are left out.

Example 2

```
SELECT name, age, course FROM Students
WHERE age >= 20 AND course = 'Python';
```

Output (1 row):

name	age	course
Sanjana	20	Python

Explanation: Several Python students exist, but only Sanjana is 20 or older. Both conditions must hold.

8. OR

What it is: `OR` returns a row if **at least one** condition is true.

Syntax

```
WHERE condition1 OR condition2
```

Example 1

```
SELECT name, course FROM Students
WHERE course = 'Java' OR course = 'Web Development';
```

Output (4 rows):

name	course
Kiran	Java
Rohan	Web Development
Arjun	Java
Vikram	Web Development

Explanation: A row passes if its course is Java *or* Web Development.

Example 2

```
SELECT name, age, marks FROM Students
WHERE marks > 90 OR age < 19;
```

Output (3 rows):

name	age	marks
Ananya	20	91
Sneha	18	58
Anjali	22	95

Explanation: Ananya and Anjali qualify by marks, Sneha qualifies by age. They need to satisfy only one condition.

Tip: use brackets when mixing AND and OR

AND is evaluated before OR. Brackets make your intention clear and change the result.

```
SELECT name, age, marks FROM Students
WHERE course = 'Python' AND (marks > 70 OR age < 19);
```

Output (3 rows):

name	age	marks
Priya	19	74
Sneha	18	58
Sanjana	20	72

9. NOT

What it is: `NOT` reverses a condition: true becomes false, and false becomes true.

Syntax

```
WHERE NOT condition
```

Example 1

```
SELECT name, course FROM Students
WHERE NOT course = 'Python';
```

Output (8 rows):

name	course
Aarav	Data Science
Ananya	Data Science
Kiran	Java
Rohan	Web Development
Divya	Data Science
Arjun	Java
Vikram	Web Development
Anjali	Data Science

Explanation: Everyone except the Python students. This is the same as `course <> 'Python'`.

Example 2

```
SELECT name, marks FROM Students
WHERE NOT (marks >= 60);
```

Output (2 rows):

name	marks
Sneha	58
Vikram	45

Explanation: NOT flips the bracketed test, so you get students with marks below 60.

10. IN

What it is: `IN` checks whether a value matches **any value in a list**. It is a short form of many OR conditions.

Syntax

```
WHERE column IN (value1, value2, value3)
```

Example 1

```
SELECT name, course FROM Students
WHERE course IN ('Java', 'Data Science');
```

Output (6 rows):

name	course
Aarav	Data Science
Ananya	Data Science
Kiran	Java
Divya	Data Science
Arjun	Java
Anjali	Data Science

Explanation: Same as `course = 'Java' OR course = 'Data Science'` but shorter and easier to read.

Example 2

```
SELECT student_id, name FROM Students
WHERE student_id IN (1, 5, 9);
```

Output (3 rows):

student_id	name
1	Aarav
5	Sneha
9	Meena

Explanation: Picks exactly the rows with those IDs. IN also works with numbers.

11. NOT IN

What it is: `NOT IN` returns rows whose value is **not** in the list.

Syntax

```
WHERE column NOT IN (value1, value2)
```

Example 1

```
SELECT name, course FROM Students
WHERE course NOT IN ('Python', 'Java');
```

Output (6 rows):

name	course
Aarav	Data Science
Ananya	Data Science
Rohan	Web Development
Divya	Data Science
Vikram	Web Development
Anjali	Data Science

Explanation: Excludes Python and Java students, leaving Data Science and Web Development.

Example 2

```
SELECT name, age FROM Students
WHERE age NOT IN (19, 20, 21);
```

Output (6 rows):

name	age
Kiran	22
Sneha	18
Rohan	23
Divya	24
Arjun	25
Anjali	22

Explanation: Returns students whose age is anything other than 19, 20 or 21.

Careful: if the list contains `NULL`, NOT IN returns no rows at all. Avoid NULLs in the list.

12. BETWEEN

What it is: `BETWEEN` checks whether a value falls inside a range. **Both end values are included.**

Syntax

```
WHERE column BETWEEN low AND high
WHERE column NOT BETWEEN low AND high
```

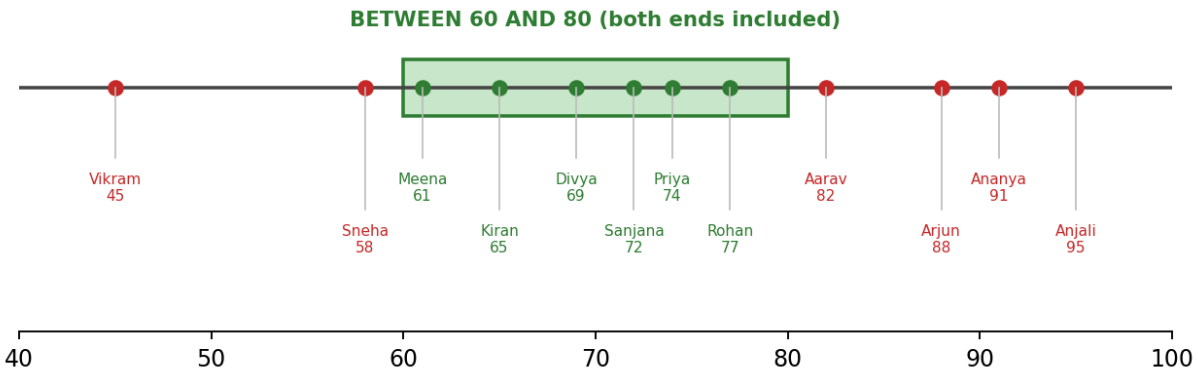


Figure 4: Marks 60 to 80 inclusive (green) vs outside the range (red)

Example 1

```
SELECT name, marks FROM Students
WHERE marks BETWEEN 60 AND 80;
```

Output (6 rows):

name	marks
Priya	74
Kiran	65
Rohan	77
Divya	69
Meena	61
Sanjana	72

Explanation: Equivalent to `marks >= 60 AND marks <= 80`. Always write the smaller value first.

Example 2

```
SELECT name, age FROM Students
WHERE age NOT BETWEEN 19 AND 22;
```

Output (4 rows):

name	age
Sneha	18
Rohan	23
Divya	24
Arjun	25

Explanation: NOT BETWEEN returns values outside the range (below 19 or above 22).

13. LIKE

What it is: `LIKE` searches for a **pattern** in text using the wildcards `%` and `_`.

Syntax

```
WHERE column LIKE 'pattern'
```

Example 1

```
SELECT name FROM Students
WHERE name LIKE 'A%';
```

Output (4 rows):

name
Aarav
Ananya
Arjun
Anjali

Explanation: Names that **start** with A. The `%` after A means "any characters can follow".

Example 2

```
SELECT name FROM Students
WHERE name LIKE '%a';
```

Output (6 rows):

name
Priya
Ananya
Sneha
Divya
Meena
Sanjana

Explanation: Names that **end** with a. The `%` before a means "anything can come before".

MySQL's default text comparison is case-insensitive, so `'a%'` and `'A%'` give the same rows.

14. NOT LIKE

What it is: `NOT LIKE` returns rows that do **not** match the pattern.

Syntax

```
WHERE column NOT LIKE 'pattern'
```

Example 1

```
SELECT name FROM Students
WHERE name NOT LIKE 'A%';
```

Output (8 rows):

name
Priya
Kiran
Sneha
Rohan
Divya
Meena
Vikram
Sanjana

Explanation: All names that do not start with A.

Example 2

```
SELECT name FROM Students
WHERE name NOT LIKE '%a';
```

Output (6 rows):

name
Aarav
Kiran
Rohan
Arjun
Vikram
Anjali

Explanation: Names that do not end with a (for example Aarav, Kiran, Arjun).

15. Wildcards % and _

Wildcard	Meaning	Example pattern	Matches
%	Zero, one or many characters	'A%'	Aarav, Anjali, Arjun ...
_	Exactly one character	'_r%'	Priya, Arjun (2nd letter r)

Pattern	Meaning	Matching names
A%	starts with A	Aarav, Ananya, Arjun, Anjali
%a	ends with a	Priya, Ananya, Sneha, Divya, Meena, Sanjana
%an%	contains 'an'	Ananya, Kiran, Rohan, Anjali, Sanjana
_r%	2nd letter is r	Priya, Arjun
A___	A + exactly 4 letters	Aarav, Arjun

Figure 5: Common LIKE patterns and the names they match in our table

Example 1

```
SELECT name FROM Students
WHERE name LIKE '%an%';
```

Output (5 rows):

name
Ananya
Kiran
Rohan
Anjali
Sanjana

Explanation: Contains the letters *an* anywhere. % on both sides means any text before and after.

Example 2

```
SELECT name FROM Students
WHERE name LIKE '_r%';
```

Output (2 rows):

name
Priya
Arjun

Explanation: _ stands for exactly one character, so the pattern means: any first letter, then *r*, then anything.

Bonus: exact length

```
SELECT name FROM Students WHERE name LIKE 'A____';   -- A + exactly 4 characters
```

Output (2 rows):

name
Aarav
Arjun

To search for a real % or _ character, escape it: `LIKE '50\%'`.

16. ORDER BY

What it is: `ORDER BY` sorts the result. Without it, MySQL does not guarantee any row order.

Syntax

```
SELECT columns FROM table_name ORDER BY column1 [ASC|DESC], column2 [ASC|DESC];
```

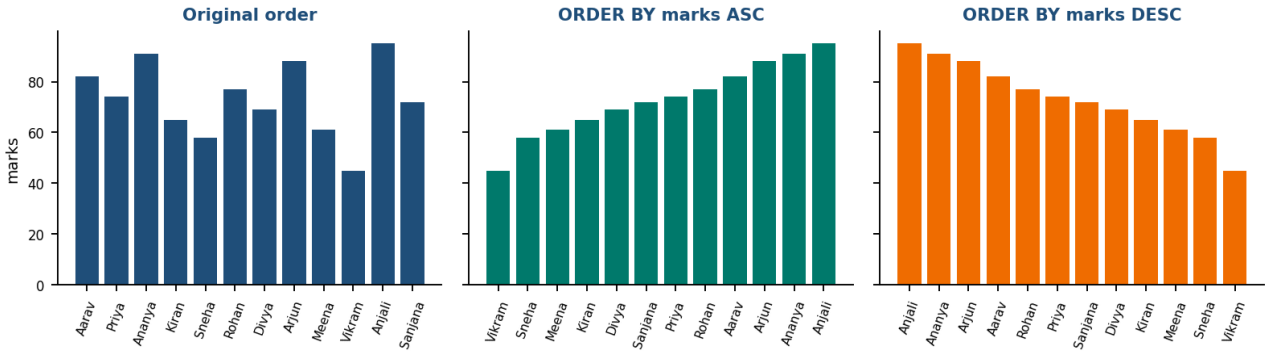


Figure 6: The same marks in original, ascending and descending order

Example 1

```
SELECT name, marks FROM Students
WHERE course = 'Python'
ORDER BY marks;
```

Output (4 rows):

name	marks
Sneha	58
Meena	61
Sanjana	72
Priya	74

Explanation: Python students sorted by marks, lowest to highest. The default direction is ascending.

Example 2

```
SELECT name, course, marks FROM Students
ORDER BY course, marks DESC;
```

Output (12 rows):

name	course	marks
Anjali	Data Science	95
Ananya	Data Science	91
Aarav	Data Science	82
Divya	Data Science	69
Arjun	Java	88
Kiran	Java	65
Priya	Python	74
Sanjana	Python	72
Meena	Python	61
Sneha	Python	58
Rohan	Web Development	77
Vikram	Web Development	45

Explanation: Sorts by *course* A to Z first. Inside each course, ties are sorted by *marks* from highest to lowest. You can sort by many columns, each with its own direction.

17. ASC (Ascending)

What it is: ASC sorts from smallest to largest, A to Z, or oldest to newest. It is the default, so writing it is optional.

Syntax

```
ORDER BY column ASC
```

Example 1

```
SELECT name, marks FROM Students
WHERE course = 'Data Science'
ORDER BY marks ASC;
```

Output (4 rows):

name	marks
Divya	69
Aarav	82
Ananya	91
Anjali	95

Explanation: Lowest marks first: 69, 82, 91, 95.

Example 2

```
SELECT name, age FROM Students
WHERE age < 20
ORDER BY name ASC;
```

Output (3 rows):

name	age
Meena	19
Priya	19
Sneha	18

Explanation: Text sorts alphabetically: Meena, Priya, Sneha.

18. DESC (Descending)

What it is: DESC sorts from largest to smallest, or Z to A. It must be written explicitly.

Syntax

```
ORDER BY column DESC
```

Example 1

```
SELECT name, age FROM Students
WHERE course = 'Java'
ORDER BY age DESC;
```

Output (2 rows):

name	age
Arjun	25
Kiran	22

Explanation: Oldest student first.

Example 2

```
SELECT name, marks FROM Students
WHERE marks > 85
ORDER BY name DESC;
```

Output (3 rows):

name	marks
Arjun	88
Anjali	95
Ananya	91

Explanation: Names in reverse alphabetical order (Z to A). Note that Arjun comes before Anjali, which comes before Ananya.

19. LIMIT

What it is: `LIMIT` restricts how many rows are returned. Combine it with ORDER BY to get "top N" or "bottom N".

Syntax

```
SELECT columns FROM table_name ORDER BY column LIMIT n;
```

Example 1

```
SELECT name, marks FROM Students
ORDER BY marks DESC
LIMIT 3;
```

Output (3 rows):

name	marks
Anjali	95
Ananya	91
Arjun	88

Explanation: Sorts highest to lowest, then keeps only the first 3 rows: the top 3 students.

Example 2

```
SELECT name, marks FROM Students
ORDER BY marks ASC
LIMIT 1;
```

Output (1 row):

name	marks
Vikram	45

Explanation: Sorts lowest first and keeps one row: the student with the lowest marks.

20. OFFSET

What it is: `OFFSET` skips a number of rows before LIMIT starts counting. It is used for **pagination**.

Syntax

```
LIMIT n OFFSET m      -- skip m rows, then return n rows
LIMIT m, n             -- shorthand (offset first, then count)
```

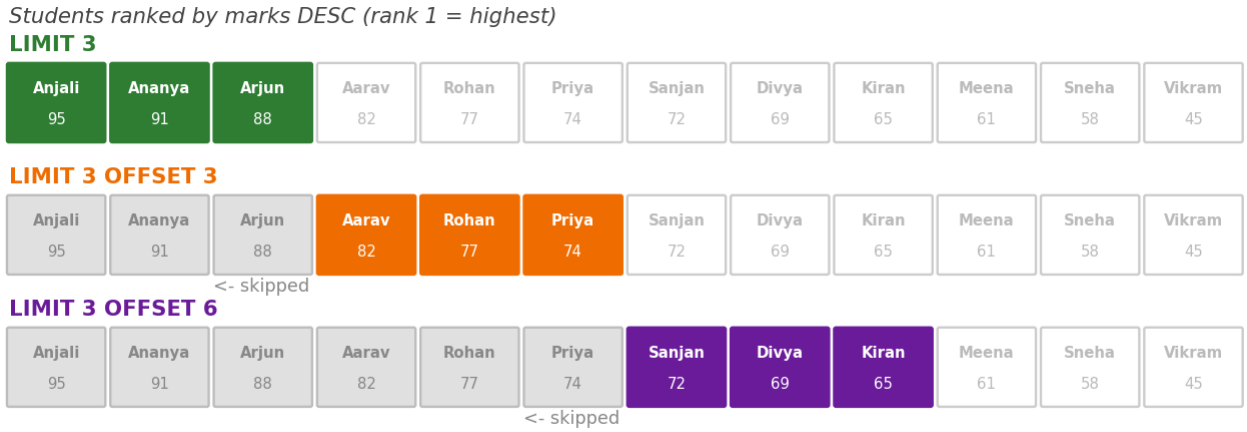


Figure 7: How LIMIT and OFFSET select a window of rows

Example 1

```
SELECT name, marks FROM Students
ORDER BY marks DESC
LIMIT 3 OFFSET 3;
```

Output (3 rows):

name	marks
Aarav	82
Rohan	77
Priya	74

Explanation: Skips the top 3 and returns the next 3 (ranks 4, 5 and 6).

Example 2

```
SELECT name, marks FROM Students
ORDER BY marks DESC
LIMIT 6, 3;
```

Output (3 rows):

name	marks
Sanjana	72
Divya	69
Kiran	65

Explanation: Shorthand form: skip 6 rows, then return 3 (ranks 7 to 9). Page formula: `OFFSET = (page_number - 1) * page_size`.

Practice Questions with Solutions

Each solution shows the query and the real output from the sample table.

Q1. Find students with marks greater than 70.

```
SELECT * FROM Students
WHERE marks > 70;
```

Output (7 rows):

student_id	name	age	course	marks
1	Aarav	21	Data Science	82
2	Priya	19	Python	74
3	Ananya	20	Data Science	91
6	Rohan	23	Web Development	77
8	Arjun	25	Java	88
11	Anjali	22	Data Science	95
12	Sanjana	20	Python	72

Q2. Find students younger than 20.

```
SELECT * FROM Students
WHERE age < 20;
```

Output (3 rows):

student_id	name	age	course	marks
2	Priya	19	Python	74
5	Sneha	18	Python	58
9	Meena	19	Python	61

Q3. Find students from Data Science.

```
SELECT * FROM Students
WHERE course = 'Data Science';
```

Output (4 rows):

student_id	name	age	course	marks
1	Aarav	21	Data Science	82
3	Ananya	20	Data Science	91
7	Divya	24	Data Science	69
11	Anjali	22	Data Science	95

Q4. Find students with marks between 60 and 80.

```
SELECT * FROM Students
WHERE marks BETWEEN 60 AND 80;
```

Output (6 rows):

student_id	name	age	course	marks
2	Priya	19	Python	74
4	Kiran	22	Java	65
6	Rohan	23	Web Development	77
7	Divya	24	Data Science	69
9	Meena	19	Python	61
12	Sanjana	20	Python	72

Q5. Find names starting with A.

```
SELECT * FROM Students
WHERE name LIKE 'A%';
```

Output (4 rows):

student_id	name	age	course	marks
1	Aarav	21	Data Science	82
3	Ananya	20	Data Science	91
8	Arjun	25	Java	88
11	Anjali	22	Data Science	95

Q6. Find names ending with a.

```
SELECT * FROM Students
WHERE name LIKE '%a';
```

Output (6 rows):

student_id	name	age	course	marks
2	Priya	19	Python	74
3	Ananya	20	Data Science	91
5	Sneha	18	Python	58
7	Divya	24	Data Science	69
9	Meena	19	Python	61
12	Sanjana	20	Python	72

Q7. Find names containing 'an'.

```
SELECT * FROM Students
WHERE name LIKE '%an%';
```

Output (5 rows):

student_id	name	age	course	marks
3	Ananya	20	Data Science	91
4	Kiran	22	Java	65
6	Rohan	23	Web Development	77
11	Anjali	22	Data Science	95
12	Sanjana	20	Python	72

Q8. Find students from Data Science or Python.

```
SELECT * FROM Students
WHERE course = 'Data Science' OR course = 'Python';
-- or: WHERE course IN ('Data Science', 'Python');
```

Output (8 rows):

student_id	name	age	course	marks
1	Aarav	21	Data Science	82
2	Priya	19	Python	74
3	Ananya	20	Data Science	91
5	Sneha	18	Python	58
7	Divya	24	Data Science	69
9	Meena	19	Python	61
11	Anjali	22	Data Science	95
12	Sanjana	20	Python	72

Q9. Find students not studying Python.

```
SELECT * FROM Students
WHERE course <> 'Python';
-- or: WHERE NOT course = 'Python';
```

Output (8 rows):

student_id	name	age	course	marks
1	Aarav	21	Data Science	82
3	Ananya	20	Data Science	91
4	Kiran	22	Java	65
6	Rohan	23	Web Development	77
7	Divya	24	Data Science	69
8	Arjun	25	Java	88
10	Vikram	21	Web Development	45
11	Anjali	22	Data Science	95

Q10. Display students in ascending order of marks.

```
SELECT * FROM Students
ORDER BY marks ASC;
```

Output (12 rows):

student_id	name	age	course	marks
10	Vikram	21	Web Development	45
5	Sneha	18	Python	58
9	Meena	19	Python	61
4	Kiran	22	Java	65
7	Divya	24	Data Science	69
12	Sanjana	20	Python	72
2	Priya	19	Python	74
6	Rohan	23	Web Development	77
1	Aarav	21	Data Science	82
8	Arjun	25	Java	88
3	Ananya	20	Data Science	91
11	Anjali	22	Data Science	95

Q11. Display students in descending order of marks.

```
SELECT * FROM Students
ORDER BY marks DESC;
```

Output (12 rows):

student_id	name	age	course	marks
11	Anjali	22	Data Science	95
3	Ananya	20	Data Science	91
8	Arjun	25	Java	88
1	Aarav	21	Data Science	82
6	Rohan	23	Web Development	77
2	Priya	19	Python	74
12	Sanjana	20	Python	72
7	Divya	24	Data Science	69
4	Kiran	22	Java	65
9	Meena	19	Python	61
5	Sneha	18	Python	58
10	Vikram	21	Web Development	45

Q12. Display the top 5 students.

```
SELECT * FROM Students
ORDER BY marks DESC
LIMIT 5;
```

Output (5 rows):

student_id	name	age	course	marks
11	Anjali	22	Data Science	95
3	Ananya	20	Data Science	91
8	Arjun	25	Java	88
1	Aarav	21	Data Science	82
6	Rohan	23	Web Development	77

Q13. Display the bottom 5 students.

```
SELECT * FROM Students
ORDER BY marks ASC
LIMIT 5;
```

Output (5 rows):

student_id	name	age	course	marks
10	Vikram	21	Web Development	45
5	Sneha	18	Python	58
9	Meena	19	Python	61
4	Kiran	22	Java	65
7	Divya	24	Data Science	69

Q14. Display unique courses.

```
SELECT DISTINCT course FROM Students;
```

Output (4 rows):

course
Data Science
Python
Java
Web Development

Q15. Find students aged between 20 and 25.

```
SELECT * FROM Students
WHERE age BETWEEN 20 AND 25;
```

Output (9 rows):

student_id	name	age	course	marks
1	Aarav	21	Data Science	82
3	Ananya	20	Data Science	91
4	Kiran	22	Java	65
6	Rohan	23	Web Development	77
7	Divya	24	Data Science	69
8	Arjun	25	Java	88
10	Vikram	21	Web Development	45
11	Anjali	22	Data Science	95
12	Sanjana	20	Python	72

Quick Reference

Goal	Clause	Example
Pick columns	SELECT	SELECT name, marks
Rename in output	AS	marks AS score
Remove duplicates	DISTINCT	SELECT DISTINCT course
Filter rows	WHERE	WHERE marks > 70
Combine conditions	AND / OR / NOT	WHERE a AND (b OR c)
Match a list	IN / NOT IN	course IN ('Java','Python')
Match a range	BETWEEN	age BETWEEN 20 AND 25
Match text pattern	LIKE / NOT LIKE	name LIKE 'A%'
Sort	ORDER BY ... ASC|DESC	ORDER BY marks DESC
Limit rows / skip rows	LIMIT / OFFSET	LIMIT 5 OFFSET 5